

Practical-6

Objective 1: Develop a program to demonstrate different types of constructor behaviour in object life cycle management.

Input:

```
#include <iostream>
using namespace std;
class BankAccount{
private:
    int accountNo;
    string holderName;
    float balance;
public:
    // Default Constructor
    BankAccount() {
        accountNo = 0;
        holderName = "Not Assigned";
        balance = 0;
        cout << "Default Constructor Called" << endl;
    }
    // Parameterized Constructor
    BankAccount(int accNo, string name, float bal)
    {
        accountNo = accNo;
        holderName = name;
        balance = bal;
        cout << "Parameterized Constructor Called" << endl;
    }
    // Copy Constructor
    BankAccount(const BankAccount &obj){
        accountNo = obj.accountNo;
        holderName = obj.holderName;
        balance = obj.balance;
        cout << "Copy Constructor Called" << endl;
    }
    // Display Function
    void display() {
        cout << "\nAccount Number : " << accountNo << endl;
        cout << "Account Holder : " << holderName << endl;
        cout << "Balance      : Rs. " << balance << endl;
    }
};

int main(){
    cout << "Creating Account 1" << endl;
    BankAccount acc1;
    acc1.display();
}
```

```
cout << "\nCreating Account 2" << endl;
BankAccount acc2(1001, "Nandini Gupta", 25000);
acc2.display();
cout << "\nCreating Account 3 as a Copy of Account 2" << endl;
BankAccount acc3(acc2);
acc3.display();
return 0;
}
```

Output:

Creating Account 1

Default Constructor Called

Account Number : 0

Account Holder : Not Assigned

Balance : Rs. 0

Creating Account 2

Parameterized Constructor Called

Account Number : 1001

Account Holder : Nandini Gupta

Balance : Rs. 25000

Creating Account 3 as a Copy of Account 2

Copy Constructor Called

Account Number : 1001

Account Holder : Nandini Gupta

Balance : Rs. 25000

Objective 2: Develop a program to demonstrate different types of destructor behaviour in object life cycle management.

Input:

```
#include <iostream>

using namespace std;

class Student{

private:

    int rollNo;

    string name;

public:

    // Constructor

    Student(int r, string n) {

        rollNo = r;

        name = n;

        cout << "Constructor Called for " << name << endl;

    }

    // Destructor

    ~Student() {

        cout << "Destructor Called for " << name << endl;

    }

};

int main(){

    cout << "Creating Student 1" << endl;

    Student s1(101, "Nandini");

    cout << "\nEntering Block" << endl;

    {

        Student s2(102, "Rahul");

        cout << "Inside Block" << endl;

    }

    cout << "\nCreating Dynamic Student" << endl;

    Student *s3 = new Student(103, "Aman");
```

```
    cout << "\nDeleting Dynamic Student" << endl;

    delete s3;

    cout << "\nEnd of main()" << endl;

    return 0;
}
```

Output:

Creating Student 1

Constructor Called for Nandini

Entering Block

Constructor Called for Rahul

Inside Block

Destructor Called for Rahul

Creating Dynamic Student

Constructor Called for Aman

Deleting Dynamic Student

Destructor Called for Aman

End of main()

Destructor Called for Nandini

Practical-8

Objective 1: Implement a program to pass object as argument to perform operations on user defined data.

Input:

```
#include <iostream>

using namespace std;

class Student{

private:

    int marks;

public:

    void getData(){

        cout << "Enter Marks: ";

        cin >> marks;

    }

    void display() {

        cout << "Marks = " << marks << endl;

    }

// Passing object as argument

    void compare(Student s)

    {

        if (marks > s.marks)

            cout << "First Student has Higher Marks." << endl;

        else if (marks < s.marks)

            cout << "Second Student has Higher Marks." << endl;

        else

            cout << "Both Students have Equal Marks." << endl;

    }

};

int main(){

    Student s1, s2;

    cout << "Enter Details of Student 1" << endl;
```

```
s1.getData();  
  
cout << "Enter Details of Student 2" << endl;  
  
s2.getData();  
  
cout << "\nStudent Details" << endl;  
  
s1.display();  
  
s2.display();  
  
cout << "\nComparison Result" << endl;  
  
s1.compare(s2);  
  
return 0;  
  
}
```

Output:

Enter Details of Student 1
Enter Marks: 85

Enter Details of Student 2
Enter Marks: 78

Student Details
Marks = 85
Marks = 78

Comparison Result
First Student has Higher Marks.